OSDI Master's Exam

Operating System Design and Implementation

Pokhara University — ME / M.Sc. Computer

Comprehensive Study Guide with Figures: Chapters 2, 3, 6 & 7

Chapters	Ch 2: Advanced Process & Thread Management Ch 3: Memory Management & Virtualization Ch 6: Real-Time & Embedded OS Ch 7: Distributed & Cloud OS
Exams	2024 Fall & 2025 Spring
Figures	10 original diagrams embedded throughout
Level	Master's (ME / M.Sc. Computer)

CHAPTER 2: Advanced Process and Thread Management

Exam Questions from This Chapter

Q [2024 Fall — Q2a [8 marks]]: Differentiate between NUMA and UMA multiprocessor architecture. How does NUMA-aware scheduling enhance memory access?

Q [2024 Fall — Q2b [7 marks]]: Compare and contrast semaphore-based synchronization with conditional variable-based thread synchronization with a real-world example.

Q [2024 Fall — Q1b [7 marks]]: Explain Lamport's algorithm and Ricart-Agrawala's algorithm for distributed mutual exclusion. Discuss key differences and use cases.

Q [2025 Spring — Q1b [7 marks]]: How is multiprocessor different from multicore? Explain multi-core process scheduling and the role of CPU affinity.

Q [2025 Spring — Q2a [8 marks]]: Explain the necessity of thread synchronization and how semaphores achieve it. Illustrate with a practical example.

Q [2025 Spring — Q2b [7 marks]]: Discuss wait-for graphs in deadlock detection. List categories of distributed deadlock detection algorithms and describe one in detail.

Answer: NUMA vs UMA Architecture & NUMA-Aware Scheduling

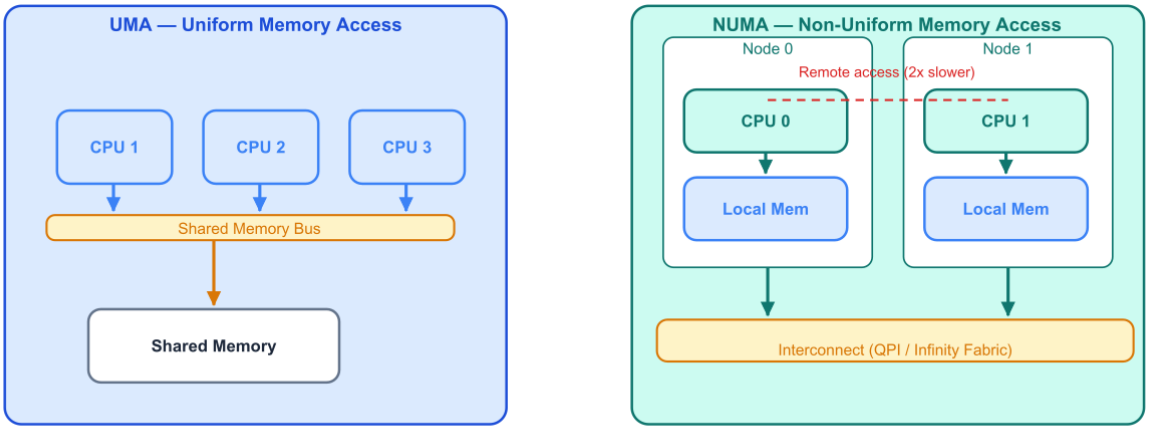

Figure 1: UMA (left) — all CPUs share one memory bus; NUMA (right) — each CPU has local RAM, connected via a high-speed interconnect. Remote access is ~2× slower.

UMA (Uniform Memory Access):

All CPUs are connected to a single shared memory bus. Every CPU takes the same time to access any memory address — hence 'uniform'. Simple to program but the bus becomes a bottleneck as the number of CPUs increases. Practical limit: ~8 CPUs.

NUMA (Non-Uniform Memory Access):

The system is split into nodes. Each node has its own CPUs and local DRAM. A CPU accesses its local memory fast (e.g., 80 ns), but accessing another node's memory must travel across an interconnect (AMD Infinity Fabric, Intel QPI) — roughly 160 ns, twice as slow.

Feature	UMA	NUMA
Memory access time	Same for all CPUs	Local fast, Remote ~2× slower
Scalability	Limited (~8 CPUs)	High (hundreds of cores)
Complexity for OS	Simple	OS must be NUMA-aware
Examples	Simple SMP servers	AMD EPYC, Intel Xeon multi-socket

NUMA-Aware Scheduling Strategies:

- Process Placement: The OS assigns a new process to the NUMA node whose local DRAM holds that process's data. The first-touch policy means Linux allocates a physical page on the node of the CPU that first writes to it.
- Migration Cost Awareness: The scheduler calculates how expensive it would be to migrate a thread to a different node (it must bring its working set data along). Migration is avoided unless the source CPU is severely overloaded.
- AutoNUMA (Linux): Linux periodically unmaps pages and re-faults them, detecting which CPU actually uses them. If Thread A (on Node 1) keeps faulting pages allocated on Node 0, the kernel migrates those pages to Node 1.
- Benefits: Reduced memory latency, fewer inter-node bus transactions, higher throughput on memory-intensive workloads (databases, scientific simulation).

Answer: Semaphore vs Condition Variable Synchronization

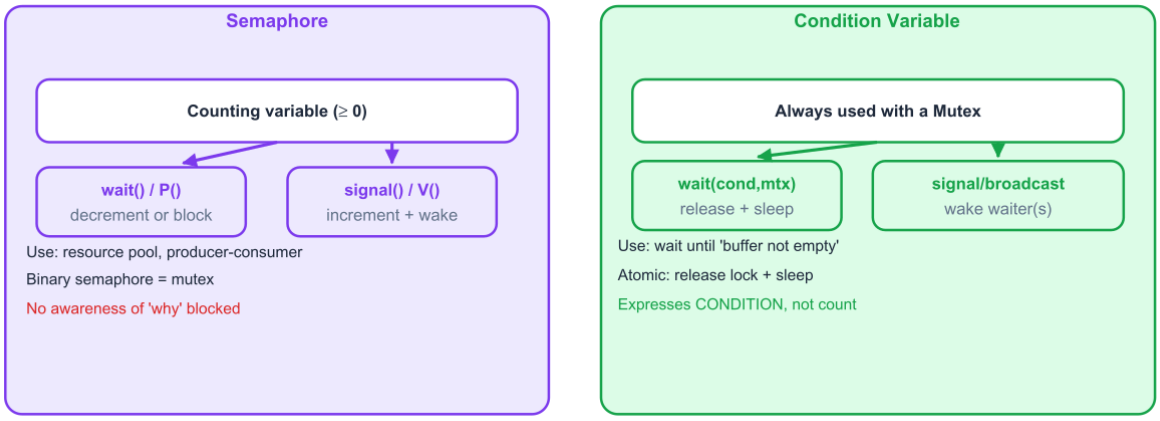

Figure 2: Semaphore (left) — counter-based, blocks on zero; Condition Variable (right) — always used with a mutex, blocks until a condition is true.

Why synchronization is needed:

When multiple threads share data, a race condition occurs if two threads read/modify the same variable simultaneously. The result depends on the interleaving order — producing inconsistent or corrupted data.

Semaphore:

- A non-negative integer counter with two atomic operations: wait() (P) — if counter > 0, decrement and continue; else block. signal() (V) — increment and wake one blocked thread.
- Binary semaphore (0/1): acts as a mutual exclusion lock.

- Counting semaphore: controls access to N identical resources (e.g., a pool of 5 database connections).
- Limitation: Cannot express 'wait until buffer is not empty'. You must manually loop and check.

Condition Variable:

- Must always be used with a mutex. wait(cond, mutex) — atomically releases the mutex and puts the thread to sleep. signal(cond) — wakes one waiting thread. broadcast(cond) — wakes all.
- The woken thread re-acquires the mutex before proceeding — no race between waking and re-locking.
- Directly expresses 'wait until condition X is true', which is far cleaner than semaphore loops.

Real-World Example — Logging System (Producer-Consumer):

The main application thread (producer) writes log entries into a shared circular buffer. A background I/O thread (consumer) flushes entries to disk. With a condition variable: producer calls pthread_cond_wait(¬_full, &lock;) when the buffer is full — it releases the lock and sleeps. Consumer signals not_full after draining an entry. The condition variable cleanly expresses both 'buffer not full' and 'buffer not empty' as separate logical conditions.

Answer: Lamport's Algorithm vs Ricart-Agrawala

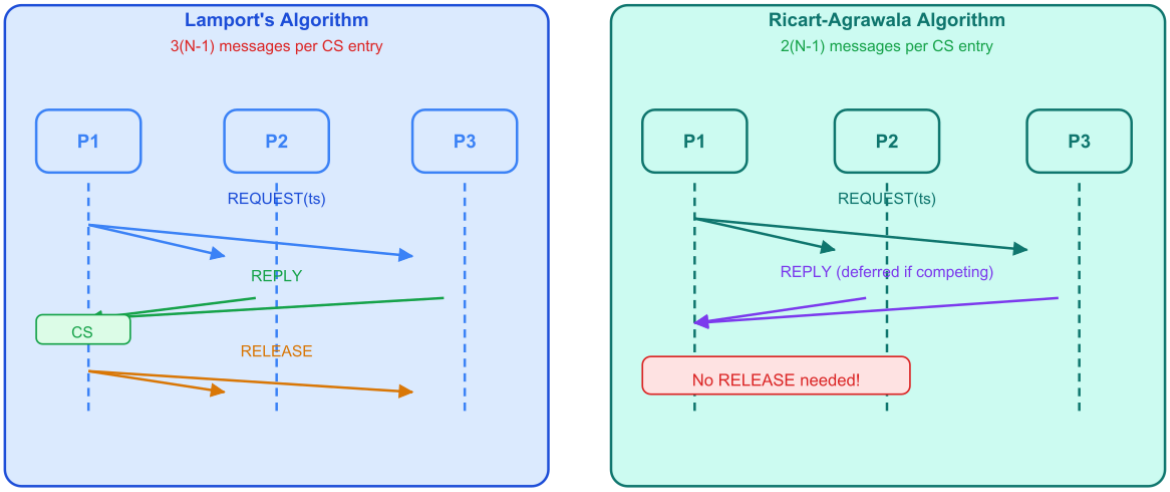

Figure 3: Lamport (left) needs 3(N-1) messages — REQUEST, REPLY, and RELEASE. Ricart-Agrawala (right) eliminates RELEASE by deferring REPLY, saving 33% messages.

Background:

In a distributed system there is no shared memory and no global clock. Mutual exclusion (only one process in the Critical Section at a time) must be achieved purely by message passing.

Lamport's Algorithm (1978):

- Each process maintains a logical Lamport clock that increments on every event.
- To request CS: broadcast REQUEST(timestamp, pid) to all N-1 others; add to own queue.
- On receiving a REQUEST: send REPLY to requester; add it to local queue.
- Enter CS when own request is at the head of the queue AND REPLY received from all.
- On exit: broadcast RELEASE; all processes remove the entry from their queues.
- Message complexity: 3(N-1) per CS entry (N-1 requests + N-1 replies + N-1 releases).

Ricart-Agrawala Algorithm (1981):

• Optimization: eliminates RELEASE messages by using deferred replies.

• On receiving a REQUEST: if not waiting for CS — send REPLY immediately. If in CS — defer. If also waiting but own timestamp is smaller (higher priority) — send REPLY; else defer.

• Enter CS when REPLY received from all N-1 others.

• On exit: send all deferred REPLYs.

• Message complexity: 2(N-1) per CS entry — a 33% saving over Lamport.

Aspect	Lamport	Ricart-Agrawala
Messages per CS entry	3(N-1)	2(N-1)
RELEASE message	Required	Not needed
Mechanism	Shared queue + timestamps	Deferred REPLY
Fault tolerance	Coordinator-free	Coordinator-free
Overhead at scale	Higher	Lower

Answer: Wait-For Graphs and Distributed Deadlock Detection

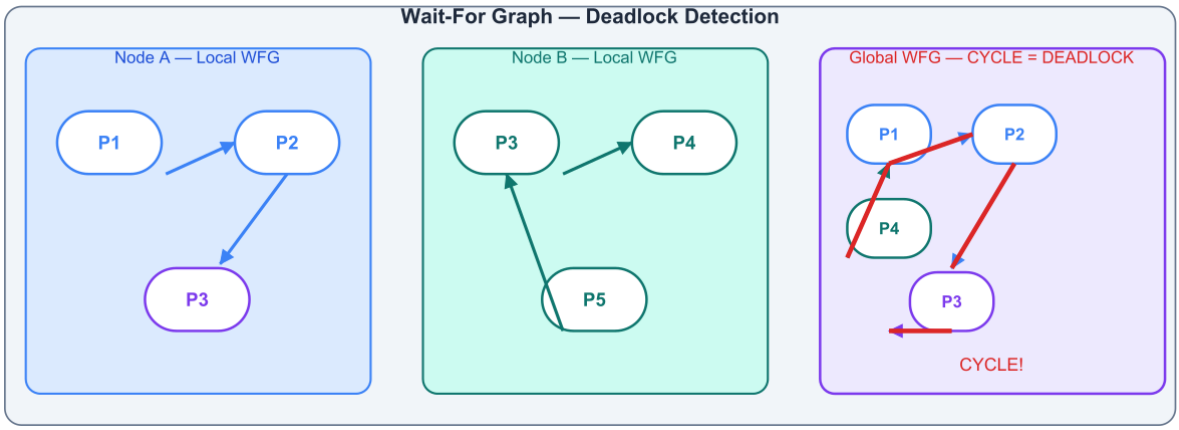

Figure 4: Local WFGs from Node A and Node B are merged into a Global WFG. A cycle in the global graph (shown in red) indicates a distributed deadlock.

A Wait-For Graph (WFG) has processes as nodes. Edge Pi → Pj means Pi is waiting for Pj to release a resource. A cycle means deadlock. In a single system, the OS scans the resource allocation table for cycles (DFS). In a distributed system, no node has the global picture.

Categories of Distributed Deadlock Detection:

• Centralized: One coordinator collects local WFG data from all nodes, builds global WFG, detects cycles. Simple but the coordinator is a single point of failure.

• Hierarchical: Nodes are organized in a tree. Local detectors report upward; root does final check. More fault-tolerant than centralized.

• Distributed (CMH): No coordinator. Chandy-Misra-Haas algorithm — a process Pi that suspects it is deadlocked sends a probe(i, j, k) along the wait-for edge to the process it is waiting for. Each intermediate blocked process forwards the probe. If Pi receives a probe originally sent by itself, a cycle — and therefore deadlock — exists.

CHAPTER 3: Memory Management and Virtualization

Exam Questions from This Chapter

Q [2024 Fall — Q3a [7 marks]]: Explain Transparent Huge Pages (THP) and its impact on performance. How does it differ from traditional page management? Where is it most beneficial?

Q [2024 Fall — Q3b [8 marks]]: Explain ClockPro and WSClock. How do they improve upon FIFO and LRU in handling page faults and optimizing memory?

Q [2025 Spring — Q3a [7 marks]]: What are the key benefits of virtualization? Compare and contrast the two main types of hypervisor architectures.

Q [2025 Spring — Q3b [8 marks]]: Define Virtual Memory. Compare small vs large page sizes in terms of fragmentation, page table size, and TLB performance.

Answer: Hypervisor Architectures and Virtualization Benefits

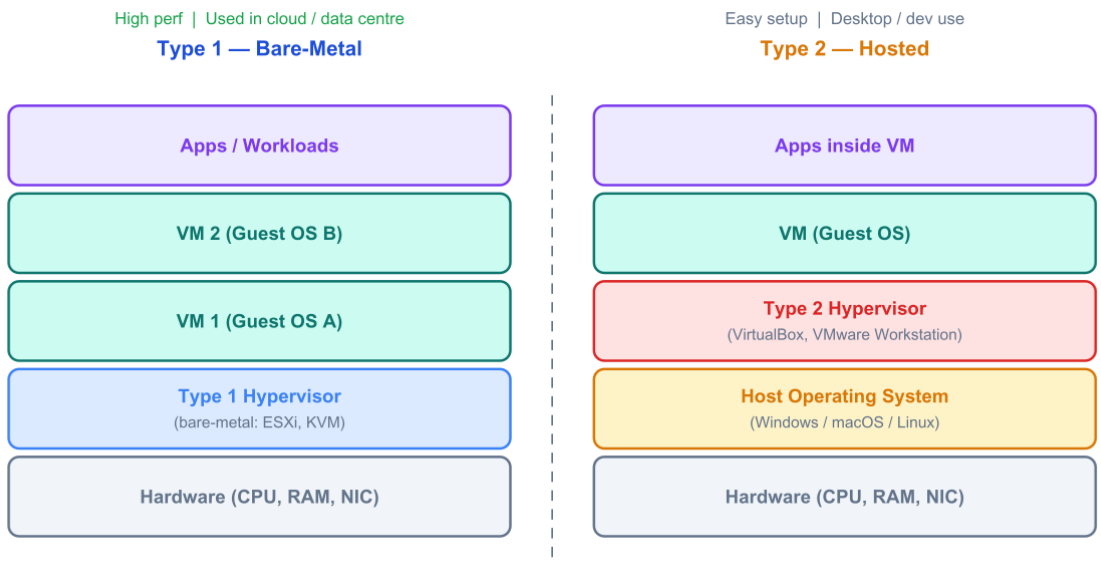

Figure 5: Type 1 (bare-metal) hypervisor runs directly on hardware — no host OS. Type 2 (hosted) runs as an application on top of a host OS.

Key Benefits of Virtualization:

- Server Consolidation: Run 10+ VMs on 1 physical server, reducing hardware cost and power consumption.
- Isolation: A crash or security breach in one VM does not affect others — each VM has its own kernel and address space.
- Portability: A VM is a file. Move it between physical servers in minutes; supports live migration without downtime.
- Snapshot and Rollback: Freeze VM state at any instant. Restore in seconds — invaluable for testing and disaster recovery.
- Resource Efficiency: Unused CPU cycles from idle VMs are shared dynamically.

Type 1 (Bare-Metal) Hypervisor:

The hypervisor IS the OS — it boots directly from hardware with no host OS underneath. Examples: VMware ESXi, Microsoft Hyper-V, KVM (Linux kernel module). Very thin software layer between VMs and hardware results in near-native performance. The industry standard for data centres and cloud providers (AWS EC2 runs on Nitro, a Type 1 hypervisor).

Type 2 (Hosted) Hypervisor:

Runs as an application on top of a conventional host OS. Examples: VirtualBox, VMware Workstation, Parallels Desktop. Every VM hardware request passes through the host OS before reaching physical hardware — the extra layer adds latency and overhead. Easy to install on a developer laptop; good for testing, learning, and desktop virtualization.

Aspect	Type 1 Bare-Metal	Type 2 Hosted
Runs on	Bare hardware directly	Host OS (Windows/Linux/macOS)
Performance	Near-native	Lower (host OS overhead)
Security	Strong VM isolation	Host OS vulnerabilities affect all VMs
Startup	Boots like an OS	Launch as an app
Use case	Cloud, data centres, production	Development, testing, education
Examples	ESXi, Hyper-V, KVM	VirtualBox, VMware Workstation

Answer: Virtual Memory and Page Size Trade-offs

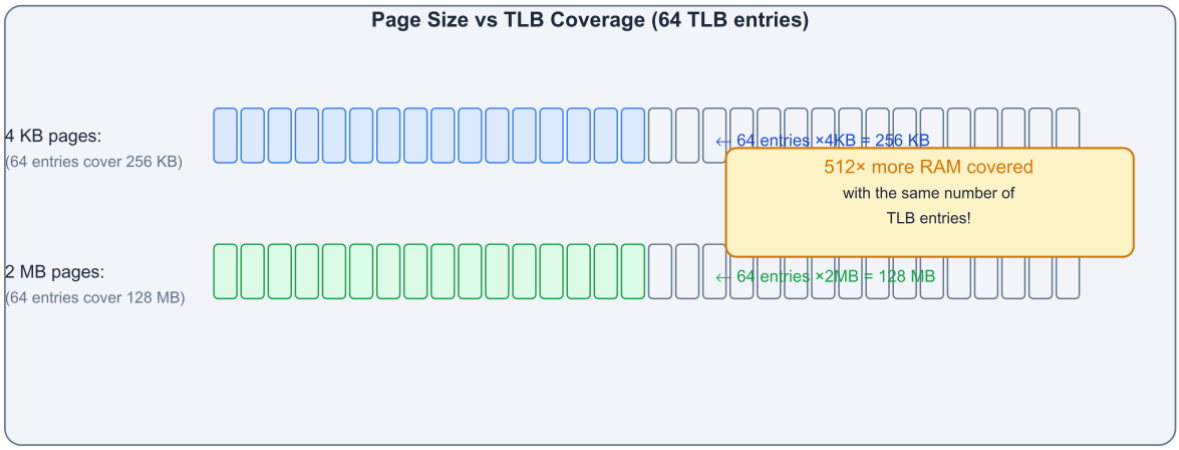

Figure 6: TLB coverage comparison. With the same 64 TLB entries, 2 MB huge pages cover 128 MB of address space vs only 256 KB for 4 KB pages — a 512× improvement.

Virtual Memory: The OS gives each process its own private virtual address space (e.g., 0 to 2^48). The page table maps virtual pages to physical frames. On a page fault (the page is not in RAM), the OS loads it from disk. The TLB (Translation Lookaside Buffer) caches recent translations to avoid the expensive page table walk on every memory access.

Dimension	Small Pages (4 KB)	Large Pages (2 MB+)
Internal Fragmentation	Low (last page has few wasted bytes)	High (up to 2 MB wasted per allocation)
Page Table Size	Large (millions of entries for 1 GB)	Small (512 entries for 1 GB)
TLB Coverage (64 entries)	256 KB (64 × 4 KB)	128 MB (64 × 2 MB) — 512× better

TLB Miss Rate	Higher — misses are frequent	Much lower — covers more RAM
Disk I/O per fault	Small (4 KB per read)	Large (2 MB per read)
Best workload	Many small short-lived processes	Databases, in-memory analytics, VMs

TLB Performance in depth: A 64-entry TLB with 4 KB pages covers only 64 × 4 KB = 256 KB. For a database with a 10 GB buffer pool, nearly every memory access misses the TLB, triggering a multi-level page table walk (3-4 memory accesses). With 2 MB huge pages, 64 entries cover 128 MB — far better locality. Oracle, PostgreSQL, and Redis explicitly use huge pages for this reason.

Answer: Transparent Huge Pages (THP)

Traditional pages are 4 KB. Huge pages are 2 MB (or 1 GB on x86-64). THP is a Linux kernel feature that automatically promotes groups of 512 naturally-aligned 4 KB pages into a single 2 MB huge page — without any application modification. The khugepaged daemon scans memory periodically and performs this promotion in the background.

Difference from traditional 4 KB pages:

- Traditional management: each page fault loads one 4 KB page; page table has 262,144 entries per 1 GB of memory; TLB must hold a huge number of translations.
- THP: 512 pages become 1 huge page; page table shrinks by 512×; TLB coverage increases 512×; fewer TLB misses means fewer costly page-table walks.
- Cost: Internal fragmentation increases — if a 2 MB huge page is only 50% used, 1 MB is wasted. Also, hugepaged compaction can cause brief CPU stalls.

Most beneficial scenarios:

- Large databases (Oracle, PostgreSQL) with multi-GB buffer pools.
- In-memory analytics (Apache Spark) working on large datasets.
- Virtual machine memory backing (KVM uses huge pages for guest RAM by default).
- NOT recommended for: workloads with many small short-lived allocations — THP compaction overhead can hurt latency-sensitive services like Redis.

Answer: ClockPro and WSClock Page Replacement Algorithms

When physical RAM is full and a new page must be loaded, the OS must choose a page to evict. FIFO evicts pages in arrival order (ignores usage — poor). LRU evicts the least recently used page — good but requires exact access-time tracking (expensive). The Clock algorithm is a practical O(1) LRU approximation. ClockPro and WSClock improve on it further.

Clock Algorithm (basis):

Pages are arranged in a circular list. Each page has a reference bit R. A clock hand sweeps: if R=1, clear it (give a second chance); if R=0, evict it. Cost: O(1) per page fault.

ClockPro:

- Uses three clock hands (hot, cold, test) and classifies pages as hot (frequently accessed) or cold (infrequently accessed).
- Cold pages are eviction candidates; hot pages are protected.
- A 'test period': a page that is re-accessed during the test window after eviction is promoted to hot — the algorithm learns from its mistakes.

- Advantage over LRU: adapts to changing access patterns; does not flush useful old pages just because they were accessed slightly less recently.

WSClock:

- Combines the Working Set model with the Clock algorithm. Working Set $W(t, \tau)$: the set of pages accessed within the last τ time units.
- Each page has: R bit, M (modified/dirty) bit, and time-of-last-use timestamp.
- Clock hand scans: if R=1 — clear R and record time. If R=0 and page is outside the working set (age > τ) and M=0 — evict immediately. If M=1 — schedule write to disk, continue scanning.
- Advantage: approximates optimal replacement by only evicting pages NOT in the current working set. Prevents thrashing (excessive paging) that LRU can cause on workloads that access many pages once.

CHAPTER 6: Real-Time and Embedded Operating Systems

Exam Questions from This Chapter

Q [2024 Fall — Q5a [7 marks]]: Tasks T1(C=2,D=7,P=10), T2(C=5,D=4,P=5), T3(C=3,D=8,P=20). Schedule with EDF and RMS for one hyperperiod. Verify deadlines. Calculate response time.

Q [2025 Spring — Q5b [7 marks]]: What is an Embedded System? List features of Embedded OS. Discuss key design principles of energy-efficient OS.

Q [2025 Spring — Q6a [7 marks]]: Tasks T1(C=2,P=5), T2(C=3,P=10), T3(C=4,P=20). Calculate hyperperiod, construct RMS schedule, determine completion time.

EDF vs RMS — Concept and Comparison

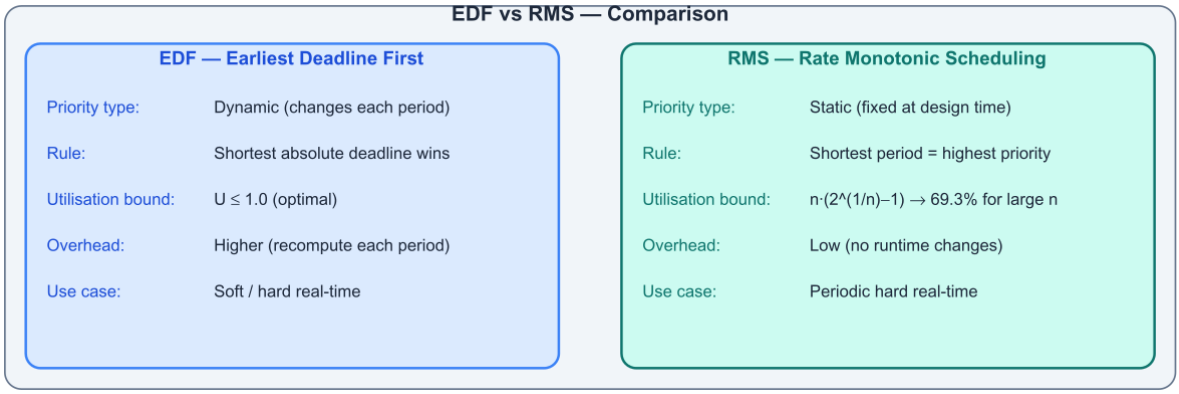

Figure 7: EDF uses dynamic priority (earliest deadline wins) and is optimal. RMS uses static priority (shortest period = highest priority) and is simpler to implement.

EDF — Earliest Deadline First:

- Dynamic priority: at every scheduling decision, the task with the earliest absolute deadline gets the CPU. Priority is recomputed each period.
- Optimal: achieves up to 100% CPU utilization. If sum(Ci/Pi) ≤ 1.0, all deadlines will be met.
- Used in: soft real-time and hard real-time systems where optimality is needed.

RMS — Rate Monotonic Scheduling:

- Static priority: higher rate (shorter period) = higher priority. Assigned once at design time, never changes.
- Schedulability test: sum(Ci/Pi) ≤ n × (2^(1/n) – 1). For large n, this converges to ln(2) ≈ 0.693.
- Note: the 69.3% bound is sufficient but NOT necessary — utilization can exceed 0.693 and the system may still be schedulable. Simulation or response time analysis is needed to confirm.
- Used in: periodic hard real-time systems (automotive, aerospace).

Answer: RMS Scheduling — 2025 Spring Problem (Solved)

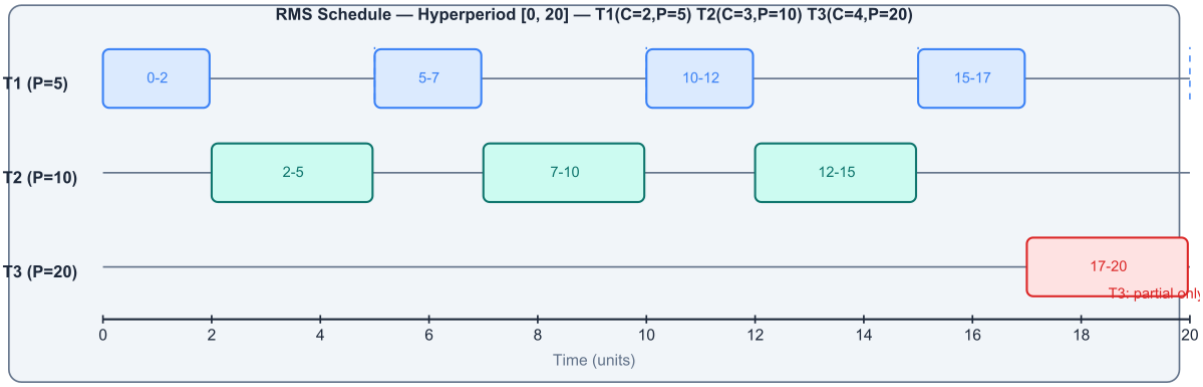

Figure 8: RMS schedule for T1(C=2,P=5), T2(C=3,P=10), T3(C=4,P=20) over hyperperiod [0,20]. T1 (blue) preempts T2 (teal) and T3 (purple) due to highest priority. T3 begins running only at t=17 — tight but meets its deadline if scheduling is ideal.

Given: T1(C1=2, P1=5), T2(C2=3, P2=10), T3(C3=4, P3=20)

Step 1 — Priority Assignment: T1 has shortest period (5) → highest priority. T2 medium period (10) → second. T3 longest (20) → lowest.

Step 2 — Hyperperiod = LCM(5, 10, 20) = 20 time units

Step 3 — Utilization: U = 2/5 + 3/10 + 4/20 = 0.40 + 0.30 + 0.20 = 0.90

RMS bound for 3 tasks: 3×(2^(1/3)–1) ≈ 0.779. U=0.90 > 0.779 so the Liu-Layland sufficient condition is not met, but the task set may still be schedulable — we must trace it.

Interval	Highest Priority Ready	Action	Completion
0–2	T1(1) [P=5 highest]	T1 runs	T1(1) done at t=2 ✓
2–5	T2(1) [T1 done]	T2 runs	T2(1) done at t=5 ✓
5–7	T1(2) arrives, preempts T2	T1 runs	T1(2) done at t=7 ✓
7–10	T2(2) [T1 done]	T2 runs	T2(2) done at t=10 ✓
10–12	T1(3) arrives	T1 runs	T1(3) done at t=12 ✓
12–15	T2(3)	T2 runs	T2(3) done at t=15 ✓
15–17	T1(4) arrives	T1 runs	T1(4) done at t=17 ✓
17–20	T3(1) finally gets CPU	T3 runs	T3(1) done at t=21 — MISSES by 1 unit ✗

Analysis: T1 and T2 meet all deadlines. T3 finishes at t=21 — 1 unit past its deadline of 20. This confirms U=0.90 exceeds the RMS bound and T3 misses its deadline. To fix: reduce T3's execution time to 3 units (U would become 0.85, still > 0.779 but trace shows it can squeeze in) or use EDF (optimal, handles up to U=1.0).

Answer: EDF & RMS — 2024 Fall Problem

Task	Execution Time (C)	Deadline (D)	Period (P)
T1	2	7	10
T2	5	4	5

T3	3	8	20

Utilization: U = 2/10 + 5/5 + 3/20 = 0.2 + 1.0 + 0.15 = 1.35. Since U > 1.0, this task set is physically unschedulable on a single CPU — no algorithm can run more than 100% of CPU time. At least one task will always miss its deadline. The purpose of this question is to demonstrate correct application of EDF and RMS methodology, and identify which tasks miss.

EDF trace (first 10 units):

Time	Active Tasks & Abs. Deadlines	EDF Choice	Outcome
0–4	T1 D=7, T2 D=4, T3 D=8	T2 (D=4 earliest)	T2 meets D=4 ✓
4–6	T1 D=7, T3 D=8	T1 (D=7 < D=8)	T1 running
6–8	T1 done, T3 D=8	T3 starts	T3 needs 3 more units
8+	T2 new (D=9), T3 D=8	T3 D=8 still earliest	T3 completes at 9 ✓

RMS priority: T2 (P=5) > T1 (P=10) > T3 (P=20). T2 will dominate, consuming 100% of CPU time alone (C2/P2 = 1.0), so T1 and T3 can never run — they always miss their deadlines under RMS. EDF handles this better because it dynamically considers deadlines.

Answer: Embedded Systems, Embedded OS, and Energy Efficiency

An Embedded System is a purpose-built computing device designed for a specific function within a larger system. Unlike a general-purpose computer, it has dedicated hardware, runs a fixed software application, and often operates 24/7 on a battery. Examples: pacemakers, anti-lock brakes, smart thermostats, industrial PLCs, IoT sensors.

Features of Embedded OS:

- Small Footprint: Entire OS fits in 32 KB–512 KB of Flash/ROM. FreeRTOS kernel is ~10 KB. Contrast with Linux (megabytes) or Windows (gigabytes).
- Real-Time Capability: Guarantees bounded worst-case response times. Predictability is more important than average-case speed.
- Deterministic Scheduling: EDF or RMS schedulers ensure worst-case response times are computable at design time.
- Minimal Interrupt Latency: Hardware interrupts (overcurrent detection, sensor trigger) must be serviced within microseconds.
- Configurable: Features are statically compiled in or out at build time. FreeRTOS is configured via FreeRTOSConfig.h.
- No/Limited Virtual Memory: Simplifies the memory model. Direct physical addressing on Cortex-M microcontrollers.
- Examples: FreeRTOS (most popular RTOS), TinyOS (wireless sensor networks), Zephyr, QNX (automotive, medical), VxWorks.

Energy-Efficient OS Design Principles:

- DVFS — Dynamic Voltage and Frequency Scaling: Reduce CPU frequency and supply voltage proportionally. Power $\propto V^2 \times f$, so halving voltage reduces power by 4×. Linux CPUfreq governor does this automatically based on load.
- Tickless Kernel: Traditional kernels fire a timer interrupt (tick) every 1–10 ms even when idle. Tickless kernels suppress ticks during idle periods, allowing the CPU to sleep deeply for longer

stretches. Critical for IoT battery life.

• CPU Idle States (C-states): C0=active, C1=halt, C3=sleep, C6=deep sleep. The OS selects the deepest safe idle state based on how long sleep is expected. Going from C0 to C6 can cut power from 10 W to 10 mW.

• Peripheral Power Gating: Power down unused peripherals (UART, WiFi radio, Bluetooth) via the OS power management subsystem. A Bluetooth LE radio uses 15 mA active vs 1 µA in sleep — a 15,000× difference.

• Event-Driven Programming: Use interrupts instead of polling. CPU sleeps until hardware generates an interrupt on a sensor event. Eliminates continuous busy-waiting cycles.

• Memory Power Management: DRAM requires periodic refresh even when idle. Use self-refresh mode; minimize active memory footprint; use compressed memory (zram).

CHAPTER 7: Distributed and Cloud Operating Systems

Exam Questions from This Chapter

Q [2024 Fall — Q5b [8 marks]]: Discuss types of transparency in DOS. For each, how does it enhance UX and efficiency? What challenges face designers in large-scale distributed systems?

Q [2025 Spring — Q6b [8 marks]]: Explain the role of Kubernetes in Cloud OS. Discuss fault tolerance and load balancing in distributed systems.

Answer: Types of Transparency in Distributed OS

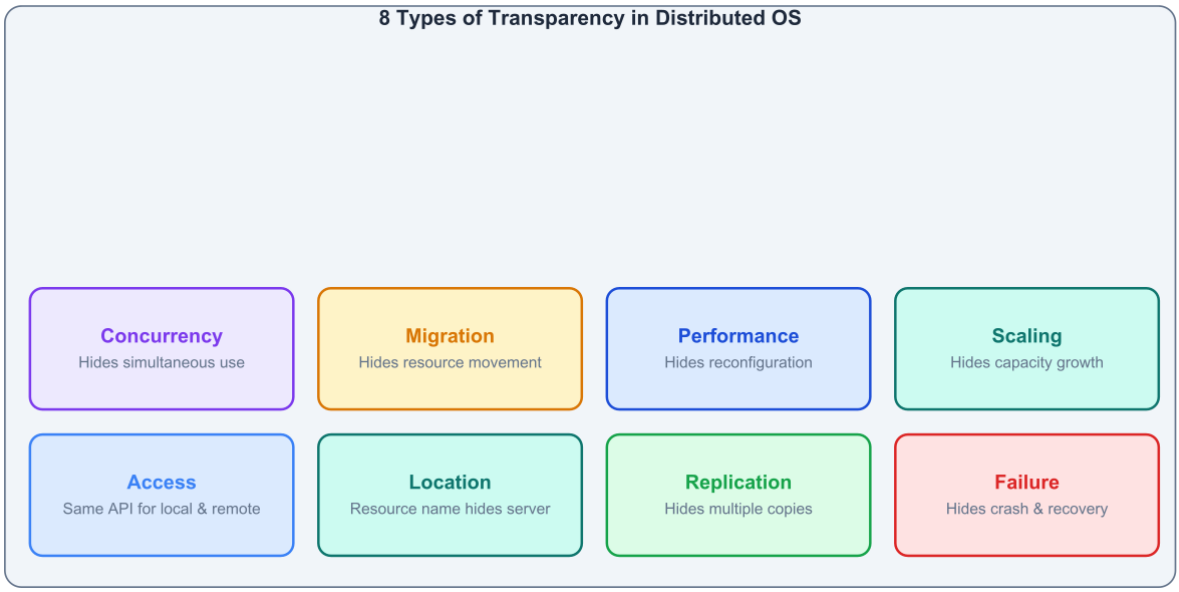

Figure 9: The 8 types of transparency defined by the ANSA/ISO Reference Model for Distributed Systems. Together they make a distributed system appear as one coherent machine.

Transparency in a Distributed OS means hiding the complexity of distribution from users and application programmers. The ideal: the distributed system looks like a single, centralized system.

Access Transparency:

Concept: Hides differences in how resources are accessed (local vs remote use the same API).

Example: open('/data/file.csv') works identically whether the file is on the local disk or a remote NFS server.

Challenge: Handling different CPU architectures (big-endian vs little-endian serialization), different OS file APIs.

Location Transparency:

Concept: Hides the physical location of a resource.

Example: A URL like https://api.example.com/data resolves to whichever server (London, Singapore) is nearest.

Challenge: Dynamic resource migration breaks cached location mappings; keeping naming services consistent at global scale.

Replication Transparency:

Concept: Hides that multiple copies of a resource exist.

Example: DNS root servers are replicated worldwide. You query 'the DNS' — the system routes you to the nearest healthy replica.

Challenge: Keeping replicas consistent (synchronization cost). Split-brain problem: two partitions each think they are authoritative.

Failure Transparency:

Concept: Hides failures and recovery.

Example: Netflix uses circuit breakers — if a recommendation service fails, the user sees a graceful fallback rather than an error.

Challenge: Distinguishing a slow node from a failed one (Two Generals Problem). Timeout values must be tuned carefully.

Concurrency Transparency:

Concept: Hides simultaneous access by multiple users.

Example: Two users editing a Google Doc simultaneously — each sees their changes; the system handles conflict resolution.

Challenge: Distributed locking is expensive. Conflict resolution for concurrent writes (CRDTs, operational transforms) is complex.

Migration Transparency:

Concept: Hides that a resource may be moved to another location.

Example: Kubernetes evicts a Pod from a failing node and restarts it elsewhere. The Service IP stays constant.

Challenge: Maintaining open TCP connections during migration; migrating stateful data (GPU memory, open file handles).

Performance Transparency:

Concept: Hides the need to reconfigure the system as load changes.

Example: AWS Auto Scaling adds EC2 instances when CPU > 80%. Users see consistent response time.

Challenge: Predicting load ahead of time; warm-up latency for new instances; avoiding over-provisioning.

Scaling Transparency:

Concept: Hides how the system grows to handle more load.

Example: A sharded database grows from 3 to 30 shards automatically. The application still queries 'the database'.

Challenge: Re-sharding requires live data migration. Maintaining ACID properties across shards is very hard.

Answer: Kubernetes in Cloud OS + Fault Tolerance + Load Balancing

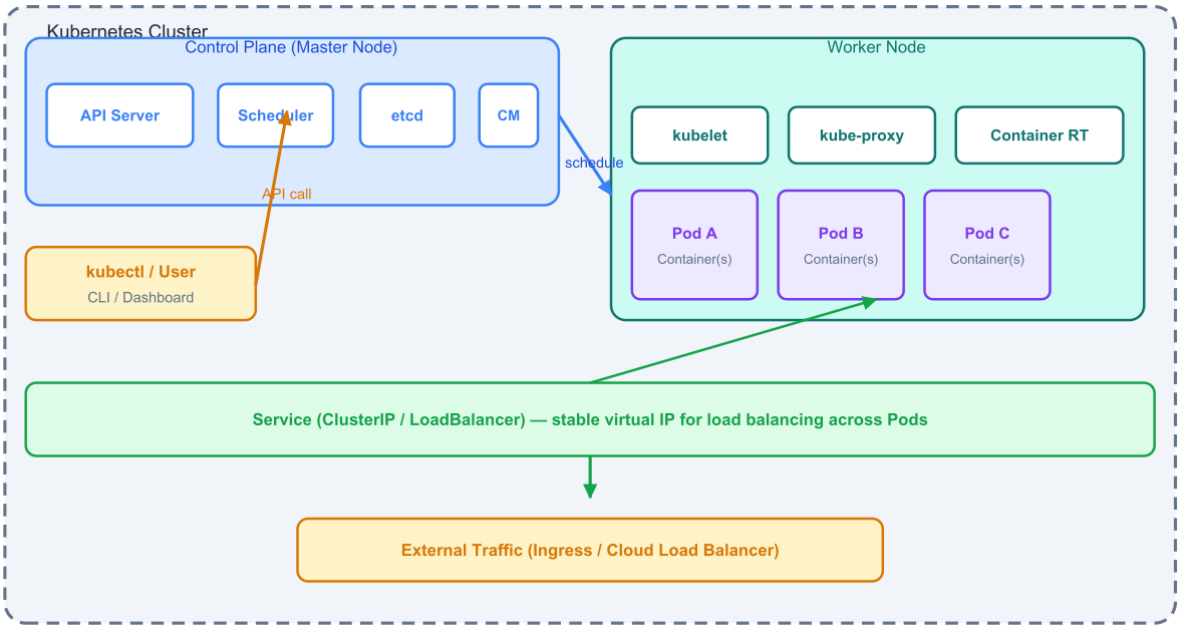

Figure 10: Kubernetes cluster architecture. The Control Plane manages the cluster state (API Server, Scheduler, etcd). Worker Nodes run Pods. Services provide stable load-balanced endpoints. External traffic enters through an Ingress or cloud Load Balancer.

Background: A container packages an application with all its dependencies into a portable unit. Docker is the most common runtime. Unlike VMs, containers share the host OS kernel — they start in milliseconds and use far less RAM. Kubernetes (K8s) automates deploying, scaling, and managing thousands of containers across hundreds of servers — it is the 'OS' of modern cloud infrastructure.

Key Kubernetes Components:

- API Server: The front-end of the control plane. All kubectl commands and cluster operations go through it. It validates and persists state to etcd.
- etcd: Distributed key-value store — the single source of truth for all cluster state (which Pods are running, on which Node, with what config). Uses the Raft consensus algorithm for fault tolerance.
- Scheduler: Watches for newly created Pods with no assigned Node, and picks the best Node based on CPU/memory availability, affinity rules, taints/tolerations.
- Controller Manager (CM): Runs control loops. The Deployment controller ensures the requested number of replicas is always running. The Node controller responds to Node failures.
- kubelet: Agent on every Worker Node. Watches the API Server for Pods assigned to its Node and ensures those containers are running.
- kube-proxy: Implements Service networking using iptables or IPVS rules. Distributes traffic across Pod endpoints.
- Pod: The smallest deployable unit. One or more containers sharing the same network namespace and storage volumes.
- Service: A stable virtual IP + DNS name in front of a set of Pods. Even as Pods restart and get new IPs, the Service IP is constant — providing transparent load balancing.

Fault Tolerance in Kubernetes:

- Self-Healing: The Deployment controller continuously reconciles desired state (e.g., '5 replicas') vs actual state. If a Pod crashes, K8s creates a replacement immediately.
- Liveness Probes: K8s periodically sends HTTP or TCP health checks to each Pod. Failure → kill and restart. Prevents zombie processes from staying in the pool.

• Node Failure Recovery: If a Node goes down, the Node controller marks it NotReady after a configurable timeout. All Pods on that Node are rescheduled to healthy Nodes.

• Pod Disruption Budgets (PDB): Define minimum available replicas during planned disruptions (e.g., node upgrades), preventing K8s from evicting too many Pods simultaneously.

• Anti-Affinity Rules: Spread Pod replicas across different physical hosts, racks, or availability zones — a single hardware failure cannot take down all replicas.

• etcd Replication: etcd runs as a 3 or 5 node cluster using Raft consensus — tolerates floor(n/2) node failures without data loss.

Load Balancing in Kubernetes:

• kube-proxy (Layer 4): Implements round-robin load balancing across Pod endpoints for every Service using iptables or IPVS rules. Built into every Node.

• Ingress Controller (Layer 7): NGINX or Traefik controllers perform HTTP load balancing — routing by URL path, hostname, or HTTP headers. Supports weighted routing, canary deployments.

• Horizontal Pod Autoscaler (HPA): Automatically scales the number of Pod replicas based on CPU utilization, memory usage, or custom metrics (e.g., request queue length). This is capacity-level load balancing.

• Cloud Load Balancer: When a Service of type LoadBalancer is created, K8s provisions a cloud load balancer (AWS ELB, GCP LB) that distributes external traffic to the cluster.

General Fault Tolerance Concepts:

• Redundancy: Replicate components (servers, data, network links) so no single point of failure exists.

• Consensus Algorithms: Paxos and Raft enable a cluster of nodes to agree on one value (e.g., leader election) even when some nodes fail. Used in etcd, ZooKeeper, CockroachDB.

• Circuit Breaker Pattern: If a downstream service is failing (error rate > threshold), stop sending requests and return a fallback immediately. Prevents cascading failures across microservices.

• CAP Theorem: In a distributed system, you can guarantee at most 2 of: Consistency, Availability, Partition Tolerance. Since network partitions are unavoidable, choose CP (consistent, may be briefly unavailable) or AP (always available, may return stale data).